

C# BACKEND TRACK | WEB API

Session 1 — Material

ASP.NET Core Web API From Console to HTTP

Project Setup · Controllers · Validation · DTOs
Illustrated with the StoreAPI — Category & Product project

Codeline Bootcamp · Backend Phase · Web API Session 1

THE BIG SHIFT — FROM CONSOLE TO HTTP

In the console application, the Presentation layer read input from **Console.ReadLine()** and wrote output with **Console.WriteLine()**. The Repository and Service layers were already correct and complete. In a Web API, you replace the Presentation layer with **Controllers**. Input now arrives as an HTTP request. Output leaves as an HTTP response in JSON. Everything else — Repositories, Services, Models — stays identical.

Console App	Web API
<code>Console.ReadLine()</code>	<code>[FromBody] / [FromRoute] / [FromQuery]</code>
<code>Console.WriteLine(result)</code>	<code>return Ok(result)</code>
<code>XxxPresentation.cs</code>	<code>XxxController.cs</code>
<code>switch/case in Main()</code>	HTTP routing (GET/POST/PUT/DELETE)
Repositories — UNCHANGED	Repositories — UNCHANGED
Services — UNCHANGED	Services — UNCHANGED

01 PROJECT SETUP

An ASP.NET Core Web API project uses **Microsoft.NET.Sdk.Web** instead of **Microsoft.NET.Sdk**. This adds the web server (Kestrel), routing engine, and middleware pipeline automatically. There is no **Main()** console loop — the web server runs continuously and handles incoming HTTP requests.

StoreAPI.csproj

```
<!-- StoreAPI.csproj — note Sdk.Web not Sdk alone -->
Sdk: Microsoft.NET.Sdk.Web
TargetFramework: net10.0
EntityFrameworkCore.SqlServer 9.0.0
EntityFrameworkCore.Tools 9.0.0
Swashbuckle.AspNetCore 7.2.0
```

New project structure:

File / Folder	Purpose
Models/	Entity classes — unchanged from console app
Data/StoreContext.cs	DbContext — unchanged
Repositories/	Data access — unchanged
Services/	Business logic — unchanged
DTOs/	Request and response shapes — new for Web API
Controllers/	Replaces Presentations/ — HTTP endpoints
Program.cs	Web server setup + DI registration (no menu loop)
appsettings.json	Connection string + configuration (replaces hardcoded string)

Program.cs — the new entry point:

Program.cs

```
var builder = WebApplication.CreateBuilder(args);  
// Register EF Core – reads connection string from appsettings.json  
builder.Services.AddDbContext<StoreContext>(options =>  
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));  
// Register repositories and services  
builder.Services.AddScoped<CategoryRepository>();  
builder.Services.AddScoped<ProductRepository>();  
builder.Services.AddScoped<CategoryService>();  
builder.Services.AddScoped<ProductService>();  
builder.Services.AddControllers();  
builder.Services.AddSwaggerGen();  
var app = builder.Build();  
app.UseSwagger(); // http://localhost:PORT/swagger  
app.UseSwaggerUI();  
app.UseAuthorization();  
app.MapControllers();  
app.Run();
```

02 CONTROLLERS

A Controller is a class that handles HTTP requests. Each public method inside it is called an **action method** and maps to a specific HTTP verb + URL route. The controller receives input from the request, calls the appropriate service, and returns an HTTP response.

The two key attributes on every controller:

<code>[ApiController]</code>	Marks the class as a Web API controller. Enables automatic model validation (400 on bad input), automatic binding of parameters from the request body, and standardised error responses.
<code>[Route("api/[controller]")]</code>	Sets the base URL for this controller. [controller] is replaced by the class name minus the word Controller — so CategoriesController maps to api/categories .

HTTP verbs and what they map to:

Attribute	HTTP Method	Operation	Returns on success
<code>[HttpGet]</code>	GET	Read / fetch	200 OK
<code>[HttpPost]</code>	POST	Create	201 Created
<code>[HttpPut]</code>	PUT	Full update	200 OK
<code>[HttpPatch]</code>	PATCH	Partial update	200 OK
<code>[HttpDelete]</code>	DELETE	Remove	204 No Content

CategoriesController — full example with all endpoint types:

Controllers/CategoriesController.cs

```
[ApiController]
[Route("api/[controller]")]
public class CategoriesController : ControllerBase
{
    private CategoryService categoryService;
    private ProductService productService;

    public CategoriesController(CategoryService categoryService, ProductService productService)
    {
        this.categoryService = categoryService;
        this.productService = productService;
    }

    // GET api/categories – returns all categories
    [HttpGet]
    public ActionResult<List<CategoryResponseDto>> GetAll()
    {
        List<CategoryResponseDto> categories = categoryService.GetAll();
        return Ok(categories);
    }

    // GET api/categories/3 – returns one category with its products
    [HttpGet("{id}")]
    public ActionResult<CategoryWithProductsDto> GetById(int id)
    {
        CategoryWithProductsDto category = categoryService.GetById(id);
        if (category == null)
            return NotFound(new { message = "Category not found." });
        return Ok(category);
    }

    // GET api/categories/3/products – nested route, returns products in category
    [HttpGet("{id}/products")]
    public ActionResult<List<ProductResponseDto>> GetProducts(int id)
    {
        if (!categoryService.Exists(id)) return NotFound(new { message = "Category not found." });
        return Ok(productService.GetByCategory(id));
    }

    // POST api/categories – creates a new category
    [HttpPost]
    public ActionResult<CategoryResponseDto> Create([FromBody] CreateCategoryDto dto)
    {
        CategoryResponseDto created = categoryService.Create(dto);
        return CreatedAtAction(nameof(GetById), new { id = created.categoryId }, created);
    }

    // PUT api/categories/3 – full update
    [HttpPut("{id}")]
```

```

public ActionResult<CategoryResponseDto> Update(int id, [FromBody] UpdateCategoryDto
dto)
{
    CategoryResponseDto updated = categoryService.Update(id, dto);
    if (updated == null) return NotFound(new { message = "Category not found." });
    return Ok(updated);
}

// DELETE api/categories/3
[HttpDelete("{id}")]
public ActionResult Delete(int id)
{
    bool deleted = categoryService.Delete(id);
    if (!deleted) return NotFound(new { message = "Category not found or has products." });
    return NoContent();
}
}

```

All endpoints in both controllers at a glance:

Method	URL	What it does
GET	api/categories	All categories with product count
GET	api/categories/{id}	One category + its products (Include)
GET	api/categories/{id}/products	Products belonging to this category
POST	api/categories	Create a new category
PUT	api/categories/{id}	Full update of a category
DELETE	api/categories/{id}	Delete category (only if no products)
GET	api/products	All products with category name
GET	api/products/{id}	One product by ID
GET	api/products/available	Only available products with stock > 0
GET	api/products/price-range?minPrice=x&maxPrice=y	Products in price range (query params)
POST	api/products	Create a new product
PUT	api/products/{id}	Full update of a product
PATCH	api/products/{id}/stock	Update stock only — auto-marks unavailable at 0
DELETE	api/products/{id}	Delete a product

03 VALIDATION

When a client sends a request body, you need to verify that the data is valid before processing it. ASP.NET Core handles this automatically through model validation — you annotate your DTO properties and the framework checks them before your action method even runs.

Key rule: You annotate the **DTO** — never the model. The model already has annotations for the database. The DTO annotations are for the HTTP request only.

How it works — `[ApiController]` does the heavy lifting:

DTOs/ProductDtos.cs + Controllers/ProductsController.cs

```
// Step 1: annotate the DTO
public class CreateProductDto
{
    [Required(ErrorMessage = "Product name is required.")]
    [MaxLength(150, ErrorMessage = "Product name cannot exceed 150 characters.")]
    public string productName { get; set; }

    [Required(ErrorMessage = "Price is required.")]
    [Range(0.01, double.MaxValue, ErrorMessage = "Price must be greater than 0.")]
    public decimal price { get; set; }

    [Required(ErrorMessage = "Category ID is required.")]
    public int categoryId { get; set; }
}

// Step 2: [ApiController] automatically returns 400 if any annotation fails
// You write NOTHING extra in the action method – it just works:
[HttpPost]
public ActionResult<ProductResponseDto> Create([FromBody] CreateProductDto dto)
{
    // if productName is missing or price <= 0, ASP.NET Core already returned 400
    // this line only runs when the request is valid
    ProductResponseDto created = productService.Create(dto);
    return CreatedAtAction(nameof(GetById), new { id = created.productId }, created);
}
```

What happens on invalid input — automatic 400 response:

Automatic 400 Bad Request response

```
// Client sends: POST api/products
{
  "productName": "", // empty – [Required] fails
  "price": -5, // negative – [Range] fails
  "categoryId": 99 // valid format but business logic checks later
}

// ASP.NET Core automatically returns:
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "productName": [ "Product name is required." ],
    "price": [ "Price must be greater than 0." ]
  }
}
```

Business validation vs model validation: [Required] and [Range] on the DTO handle format rules automatically. Business rules — like checking if the categoryId actually exists in the database — are handled manually inside the controller or service, then you return **BadRequest()** with a descriptive message yourself.

04 DTOs — DATA TRANSFER OBJECTS

A DTO is a plain class that defines the exact shape of data going into or coming out of an endpoint. You never expose a raw model class directly through an API — models are for the database, DTOs are for the HTTP layer.

Why not return the Model directly?	DTO solves this by:
• Model may contain sensitive fields (passwordHash, internal IDs) • Model with navigation properties causes circular JSON serialization errors • Client gets fields it doesn't need — wastes bandwidth • Client can set fields it shouldn't (createdAt, isAvailable) • Database schema changes break the API contract immediately	• Includes only the fields the client needs to see • No circular references — only primitive types and nested DTOs • Request DTO controls exactly what the client is allowed to send • API contract stays stable even if the model changes internally • You can compute extra fields (categoryName, productCount)

Three DTO types in the StoreAPI project:

DTOs/ProductDtos.cs

```
// 1 – Request DTO: what the client sends when CREATING a product
// Client only provides fields they should control
public class CreateProductDto
{
    public string productName { get; set; } // user provides
    public string description { get; set; } // user provides
    public decimal price { get; set; } // user provides
    public int stockQuantity { get; set; } // user provides
    public int categoryId { get; set; } // user provides
    // NO productId – system generates it
    // NO createdAt – system sets it to DateTime.Now
    // NO isAvailable – always starts as true
}

// 2 – Request DTO: what the client sends when UPDATING stock only
public class UpdateStockDto
{
    public int stockQuantity { get; set; }
    // Only one field – client cannot accidentally change the price or name
}

// 3 – Response DTO: what the API sends back after any product operation
public class ProductResponseDto
{
    public int productId { get; set; }
    public string productName { get; set; }
    public decimal price { get; set; }
    public int stockQuantity { get; set; }
    public bool isAvailable { get; set; }
    public DateTime createdAt { get; set; }
    public int categoryId { get; set; }
    public string categoryName { get; set; } // computed – joined from Category
    // NO navigation property object – no circular reference risk
}
```

Manual mapping — from Model to DTO inside the Service:

Services/ProductService.cs — mapping model to DTO

```
// In ProductService.cs — MapToDto helper converts the entity to the response shape
private ProductResponseDto MapToDto(Product product)
{
    ProductResponseDto dto = new ProductResponseDto();
    dto.productId = product.productId;
    dto.productName = product.productName;
    dto.price = product.price;
    dto.stockQuantity = product.stockQuantity;
    dto.isAvailable = product.isAvailable;
    dto.createdAt = product.createdAt;
    dto.categoryId = product.categoryId;
    // category name comes from the navigation property loaded by Include()
    dto.categoryName = product.Category != null ? product.Category.categoryName : "";
    return dto;
}
```

Compound service example — DeactivateProductsByCategory: When a category is deactivated, all its products must be marked unavailable too. This cross-entity business rule lives in **ProductService** and is called from the controller. The repository loads the products with **GetByCategory()**, the service loops and sets **isAvailable = false** on each, then calls **productRepo.Update()** once to save all changes. No Console. No database calls directly in the controller.